BACKEND ENGINEERING

DAY 6

Connecting FastAPI with PostgreSQL

A practical guide to database setup, SQLAlchemy 2.x, database models, Pydantic schemas, sessions, CRUD APIs, and testing.

Learning sequence
Days 1–4: FastAPI foundations and JSON/MongoDB practice
Day 5: Relational database and SQL fundamentals
Day 6: Connect FastAPI to PostgreSQL and build persistent CRUD

1. Learning outcomes

• Understand how a FastAPI request reaches PostgreSQL and returns a response.

• Create a PostgreSQL database and application user.

• Install SQLAlchemy 2.x and the Psycopg 3 driver.

• Separate database configuration, SQLAlchemy models, Pydantic schemas, and routes.

• Use a database session safely with FastAPI dependency injection.

• Build Create, Read, Update, and Delete endpoints.

• Test success cases, validation errors, duplicate IDs, and missing records.

Prerequisites: FastAPI routes, Pydantic BaseModel, HTTP methods, Python functions, and Day 5 SQL basics.

Roadmap note: This lesson covers PostgreSQL as your relational option. MongoDB remains your main NoSQL learning track; the separation of routes, schemas, and database logic applies to both.

2. Understand the request flow

```
Client / Swagger UI
       |
       v
FastAPI route  --->  Pydantic input schema
       |
       v
SQLAlchemy Session  --->  PostgreSQL
       |
       v
Pydantic response schema  --->  JSON response
```

• Route: receives the HTTP request and decides what operation to perform.

• Schema: validates request data and defines the public response shape.

• Database model: describes a database table using SQLAlchemy.

• Session: manages database operations and transactions for a unit of work.

• PostgreSQL: permanently stores the records.

3. Install packages

Activate your virtual environment, then run:

```
python -m pip install fastapi uvicorn sqlalchemy psycopg[binary] python-dotenv
```

FastAPI builds the API; Uvicorn runs it; SQLAlchemy is the ORM/database toolkit; Psycopg connects Python to PostgreSQL; python-dotenv loads local environment variables.

```
python -m pip show fastapi sqlalchemy psycopg
```

If installation fails for your Python version, check the package's current compatibility notes and use a supported Python environment rather than mixing packages.

4. Create a PostgreSQL database

Install PostgreSQL and start its service. Use pgAdmin or psql. Run as a PostgreSQL administrator, replacing the example password:

```
CREATE USER fastapi_user WITH PASSWORD 'replace_with_a_strong_password';
CREATE DATABASE hospital_db OWNER fastapi_user;
```

The application will connect to hospital_db using fastapi_user. Keep credentials private.

5. Project structure

```
fastapi_postgres/
|-- main.py
```

```
|-- database.py
|-- models.py
|-- schemas.py
|-- .env
|-- .gitignore
```

Keep these files in one folder for this first exercise. Add .env to .gitignore so credentials are not committed to Git.

6. Configure the connection — database.py

Create a .env file:

```
DATABASE_URL=postgresql+psycopg://fastapi_user:replace_with_a_strong_password@localhost:5432/hospital_db
```

Create database.py:

```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

load_dotenv()
DATABASE_URL = os.getenv('DATABASE_URL')
if not DATABASE_URL:
    raise RuntimeError('DATABASE_URL is not configured')

engine = create_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Concepts: engine manages database connectivity and pooling. SessionLocal creates sessions. get_db() gives a request a session and closes it afterwards. A session is not the database itself.

7. Create the SQLAlchemy table model — models.py

```
from sqlalchemy import String, Integer
from sqlalchemy.orm import Mapped, mapped_column
from database import Base

class Doctor(Base):
    __tablename__ = 'doctors'

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    uid: Mapped[str] = mapped_column(String(20), unique=True, index=True)
    name: Mapped[str] = mapped_column(String(20))
    age: Mapped[int] = mapped_column(Integer)
    dept: Mapped[str] = mapped_column(String(50))
    branch: Mapped[str] = mapped_column(String(50))
    phno: Mapped[str] = mapped_column(String(20))
    address: Mapped[str] = mapped_column(String(200))
```

The id is the database primary key. uid is your business identifier and is unique. Store phone numbers as strings because they can contain leading zeroes or a plus sign.

8. Create Pydantic schemas — schemas.py

```
from pydantic import BaseModel, Field, ConfigDict

class DoctorCreate(BaseModel):
    uid: str = Field(min_length=3, max_length=20)
    name: str = Field(min_length=1, max_length=20)
    age: int = Field(gt=0)
    dept: str
    branch: str
    phno: str
    address: str

class DoctorUpdate(BaseModel):
    name: str = Field(min_length=1, max_length=20)
```

```
    age: int = Field(gt=0)
    dept: str
    branch: str
    phno: str
    address: str

class DoctorResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    uid: str
    name: str
    age: int
    dept: str
    branch: str
    phno: str
    address: str
```

Model vs schema: the SQLAlchemy model describes how data is stored in a table. The Pydantic schema validates what the API accepts and returns. The response schema also prevents private database fields from being exposed.

9. Create the table — main.py

```
from fastapi import FastAPI
from database import engine, Base
import models

app = FastAPI()

@app.on_event('startup')
def create_tables():
    Base.metadata.create_all(bind=engine)
```

Import models before create_all so SQLAlchemy knows about the Doctor table. This automatic table creation is acceptable for this beginner exercise. For production schema changes, learn Alembic migrations.

10. Add CRUD routes — main.py

Add these imports and routes below the startup code:

```
from fastapi import Depends, HTTPException
from sqlalchemy.orm import Session
from sqlalchemy import select
from database import get_db
from models import Doctor
from schemas import DoctorCreate, DoctorUpdate, DoctorResponse

@app.post('/doctors', response_model=DoctorResponse, status_code=201)
def create_doctor(data: DoctorCreate, db: Session = Depends(get_db)):
    existing = db.scalar(select(Doctor).where(Doctor.uid == data.uid))
    if existing:
        raise HTTPException(status_code=409, detail='Doctor already exists')
    doctor = Doctor(**data.model_dump())
    db.add(doctor)
    db.commit()
    db.refresh(doctor)
    return doctor

@app.get('/doctors/{doc_id}', response_model=DoctorResponse)
def get_doctor(doc_id: int, db: Session = Depends(get_db)):
    doctor = db.get(Doctor, doc_id)
    if doctor is None:
        raise HTTPException(status_code=404, detail='Doctor not found')
    return doctor

@app.put('/doctors/{doc_id}', response_model=DoctorResponse)
def update_doctor(doc_id: int, data: DoctorUpdate, db: Session = Depends(get_db)):
    doctor = db.get(Doctor, doc_id)
    if doctor is None:
        raise HTTPException(status_code=404, detail='Doctor not found')
    for field, value in data.model_dump().items():
        setattr(doctor, field, value)
    db.commit()
    db.refresh(doctor)
    return doctor

@app.delete('/doctors/{doc_id}')
def delete_doctor(doc_id: int, db: Session = Depends(get_db)):
    doctor = db.get(Doctor, doc_id)
    if doctor is None:
        raise HTTPException(status_code=404, detail='Doctor not found')
```

```
    db.delete(doctor)
    db.commit()
    return {'message': 'Doctor deleted successfully'}
```

This example uses the numeric database primary key id in the URL. The separate uid remains unique and can be used for a lookup route later.

11. Understand key SQLAlchemy operations

Operation	Purpose	Example
select(...)	Build a SELECT query	select(Doctor)
db.scalar(...)	Return one scalar result	db.scalar(select(...))
db.add(obj)	Stage insertion	db.add(doctor)
db.commit()	Commit transaction	db.commit()
db.refresh(obj)	Reload generated values	db.refresh(doctor)
db.get(Model, id)	Find by primary key	db.get(Doctor, doc_id)
db.delete(obj)	Stage deletion	db.delete(doctor)

12. Run and test

```
uvicorn main:app --reload
```

Open http://127.0.0.1:8000/docs and test:

• POST /doctors — create a doctor.

• GET /doctors/1 — retrieve using the actual id returned by POST.

• PUT /doctors/1 — update all editable fields using the full request body.

• DELETE /doctors/1 — delete the record.

• GET /doctors/1 again — confirm the API returns HTTP 404.

Example POST JSON:

```
{
  "uid": "DOC101",
  "name": "Ravi",
  "age": 35,
  "dept": "Cardiology",
  "branch": "Main",
  "phno": "9876543210",
  "address": "Guntur"
}
```

13. Common errors

• ModuleNotFoundError: activate the correct virtual environment and install with that environment's Python.

• Connection refused: check that PostgreSQL is running and the host/port are correct.

• Password authentication failed: verify credentials in .env.

• Database does not exist: create hospital_db and check DATABASE_URL.

• Relation does not exist: confirm the model was imported before create_all.

• 422 Unprocessable Entity: compare JSON with the Pydantic schema.

• 409 Conflict: the UID already exists; use another UID.

• 500 Internal Server Error: read the complete terminal traceback.

14. Security and good practices

• Keep database credentials in environment variables; do not hardcode them.

• Add .env to .gitignore and never publish real credentials.

• Never return passwords or password hashes in public response schemas.

• Use database constraints such as primary keys and unique constraints.

• Use Alembic migrations, tests, logging, and robust transaction handling in production.

• Add authentication and authorization before exposing doctor or patient records.

15. Hospital API practice lab

• Build the project files and connect FastAPI to your local PostgreSQL database.

• Create two doctors with different UIDs and verify both rows in pgAdmin or psql.

• Try age 0, a short UID, and a duplicate UID; observe validation or conflict responses.

• Implement GET all doctors and GET by UID.

• Explain why the SQLAlchemy model and Pydantic schema are separate.

16. Review questions

• What is the difference between PostgreSQL and SQLAlchemy?

• What is a database engine?

• What is a database session, and why should it be closed?

• What is the difference between db.add() and db.commit()?

• Why does a table need a primary key?

• Why is uid marked unique?

• What does response_model do in FastAPI?

• Why should passwords never appear in DoctorResponse?

• What is the difference between PUT and PATCH?

• Why are migrations useful?

17. Day 6 completion checklist

• ■ I created a PostgreSQL database and application user.

• ■ I installed SQLAlchemy and Psycopg in my virtual environment.

• ■ I configured the connection using .env.

• ■ I understand engine, session, and dependency injection.

• ■ I created a SQLAlchemy Doctor model.

• ■ I created Pydantic create, update, and response schemas.

• ■ I implemented POST, GET, PUT, and DELETE.

• ■ I tested success and error cases in /docs.

• ■ I can explain model vs schema in my own words.

18. Official documentation

- FastAPI SQL databases: https://fastapi.tiangolo.com/tutorial/sql-databases/
- SQLAlchemy 2.0 ORM quick start: https://docs.sqlalchemy.org/en/20/orm/quickstart.html
- Psycopg 3: https://www.psycopg.org/psycopg3/docs/
- PostgreSQL: https://www.postgresql.org/docs/
- Pydantic models: https://docs.pydantic.dev/latest/concepts/models/

Suggested Day 7: Connect MongoDB to FastAPI and implement the same Doctor CRUD API. Then compare MongoDB and PostgreSQL so MongoDB remains your main database track and PostgreSQL your relational alternative.